



LLM活用入門から応用まで： プロンプト設計・LoRA・RAG

ヒューマンインタフェースシンポジウム 2026

ファムフォーロン、三林亮太

自己紹介



基本情報

- ファムフーロン
- 筑波大学 図書館情報メディア系 特任助教

研究分野

- 情報検索、機械学習

自己紹介



三林 亮太
(Ryota Mibayashi)

所属：神戸大学 国際文化学研究科 助教



産業技術総合研究所 人工知能センター 招聘研究員



専門分野：自然言語処理、音楽情報処理

所属学会：日本データベース学会、情報処理学会

これまでの経歴等：

- 2026年6月 京都未踏的人材発掘・育成事業 PM
- 2025年7月 覚醒プロジェクト
- 2022年6月 IPA未踏IT人材発掘・育成事業 未踏スーパークリエイター

京都未踏

京都未踏的人材発掘・育成プログラム

覚醒
プロジェクト



本チュートリアル概要

- ◆ LLMの基本から、プロンプト設計、RAG、LoRAによるファインチューニングまでをハンズオンで学びます。
- ◆ 各パートで理論と実践をバランスよく学習。
- ◆ 最終的には自分のデータやタスクに合わせたLLM活用力を身につけることを目指します。

キーワード

Prompt

RAG

ReAct

LoRA

BERT

GPT-2

アジェンダ

1.Introduction

2.プロンプト設計

3.RAG(ハンズオンあり)

4.ReAct

5.機械学習の基礎知識

6.LoRA(ハンズオンあり)

7.まとめ

本日の教材

<https://sloth-babies.github.io/hi2026-tutorial-llm/>



HI2026 LLM チュートリアル

Google

+ HI2026 LLM チュートリアル

AI モード →

Google 検索

I'm Feeling Lucky



GitHub Pages documentation

<https://sloth-babies.github.io/hi2026-tutorial-llm/> :

LLM活用入門から応用まで：プロンプト設計・LoRA・RAG

この講習会では、大規模言語モデル（LLM）の基本から、出力を改善するプロンプト設計、外部データを活用するRAG、モデルを適応させるLoRAまでを扱います。各パートで考え方 ...

本日の教材

<https://sloth-babies.github.io/hi2026-tutorial-llm/>

03 / RESOURCES

演習・資料

パスワード入力後に、RAGとLoRAの演習リンクを表示できます。

PRIVATE MATERIALS

ハンズオン資料を表示

パスワードを入力すると、RAGとLoRAのColabリンクが表示されます。
このパスワードは画面上の表示を切り替えるためのものです。

パスワード

パスワードを入力

リンクを表示

HANDS-ON

RAGハンズオンColab

検索拡張生成の仕組みを、ノートブックで手を動かしながら確認します。
パスワードで表示

HANDS-ON

LoRAハンズオンColab

LoRAによるモデル適応を、ノートブックで実際に試しながら学びます。
パスワードで表示

REFERENCE

講義資料 (PDF)

コース3の講義スライドをダウンロードできます。
TBA

PW: hi2026

1

Introduction

LLMとは何かを理解し、
本チュートリアル全体像を把握する

LLMとは？

- ◆ Large Language Model（大規模言語モデル）の略
- ◆ 大量のテキストデータを学習し、
人間のよう自然な文脈を生成・理解できるモデル
- ◆ 代表例：GPT、Llama、Gemma、Claudeなど



LLMでできること



文章生成・要約・翻訳・質問応答



アイデア出し・ブレインストーミング



プログラミング支援・コード生成

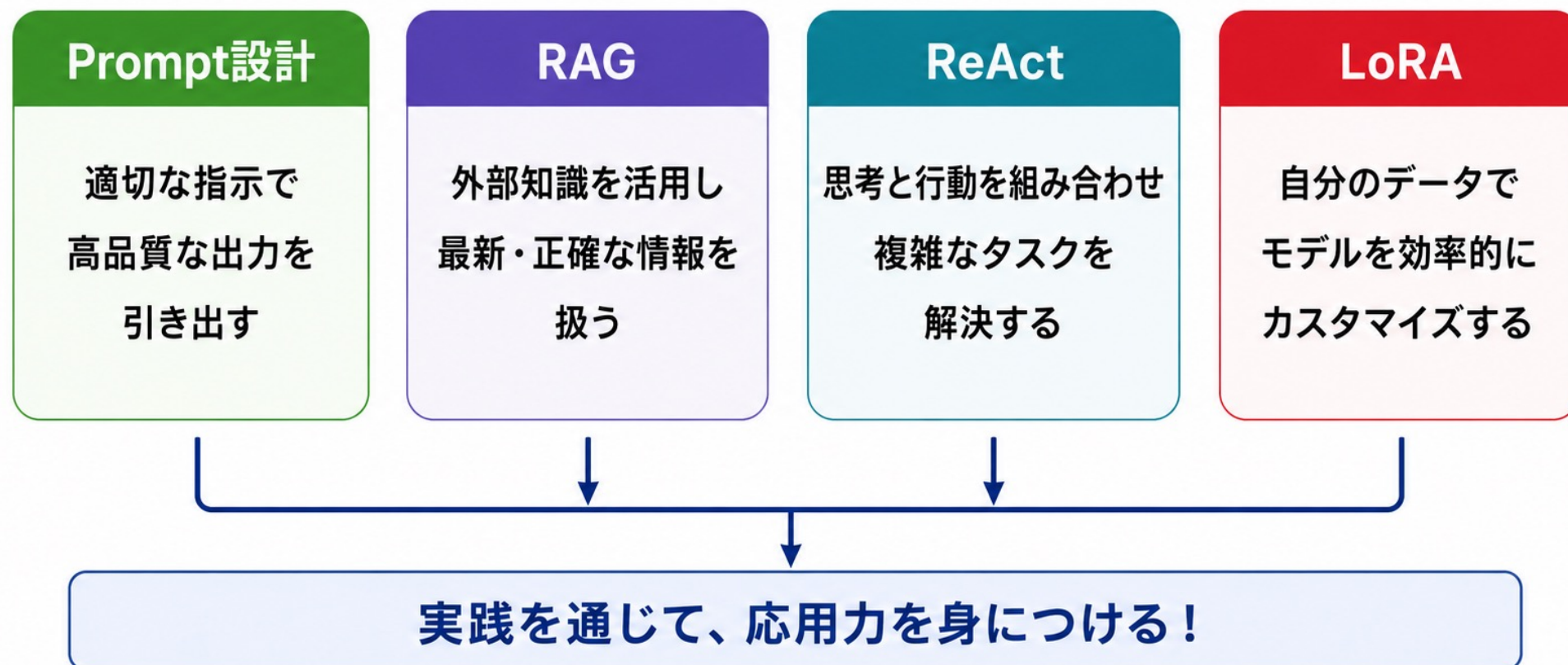


データ分析や説明



業務の自動化・効率化 など

本チュートリアルで学ぶこと



2

Prompt設計

様々なPromptの作り方

プロンプトとは？

- ◆ LLMに対する**指示文**や**質問文**のこと
- ◆ プロンプトの書き方次第で、出力の**質が大きく変わる**
- ◆ **明確**で具体的な指示が**重要**

悪い例

日本について教えて。

良い例

日本の首都はどこですか？人口は？
簡潔に2-3行で教えてください。

プロンプト設計の代表的手法

Zero-Shot: 例を与えずに指示する

One-Shot: 1つの例を与える

Few-Shot: 複数の例を与える

Chain of Thought (CoT): 思考のステップを明示する

ReAct: 思考と行動を組み合わせる

RAG: 外部知識を参照する

Zero-Shot

例を与えずに、直接質問や指示をする方法

Q: 日本首都はどこですか？



Few-shot（複数例を与える）

複数の例を与えることで、より安定した出力に

Q: フランスの首都はどこですか？

A: パリ

Q: ドイツの首都はどこですか？

A: ベルリン

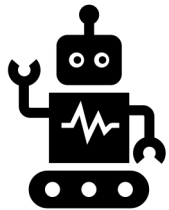
Q: 日本の首都はどこですか？



CoT (Chain of Thought)

思考のステップを明示することで、複雑な問題の精度が向上

Aの商品は1,200円で20%オフ、Bの商品は1,000円で10%オフです。どちらが安いかわかる順番に計算してください。



A: $1,200 \times 0.8 = 960$ 円

B: $1,000 \times 0.9 = 900$ 円

Bのほうが60円安いです。

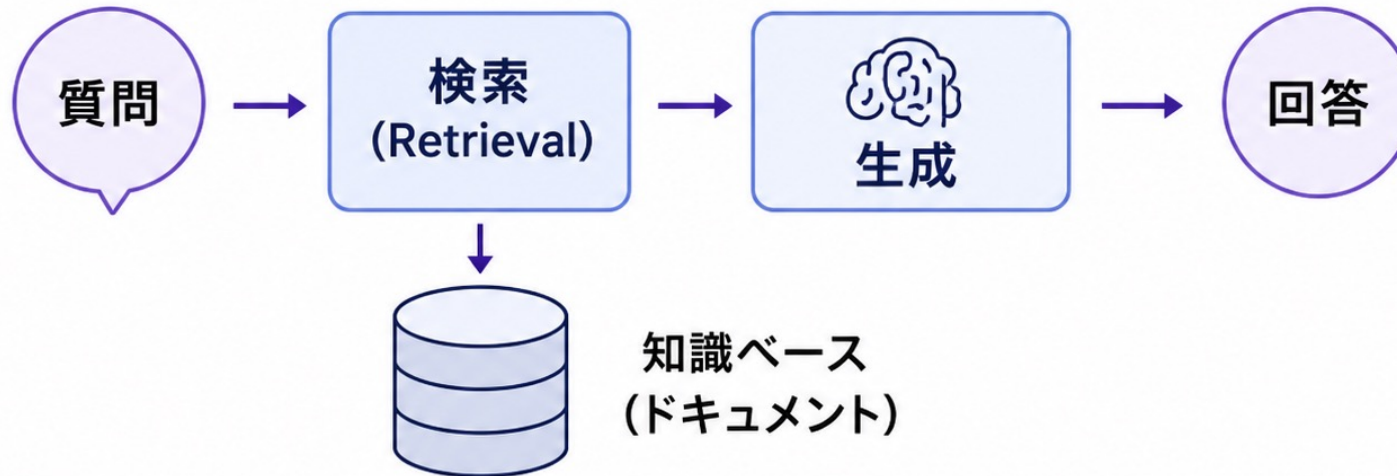
3

RAG

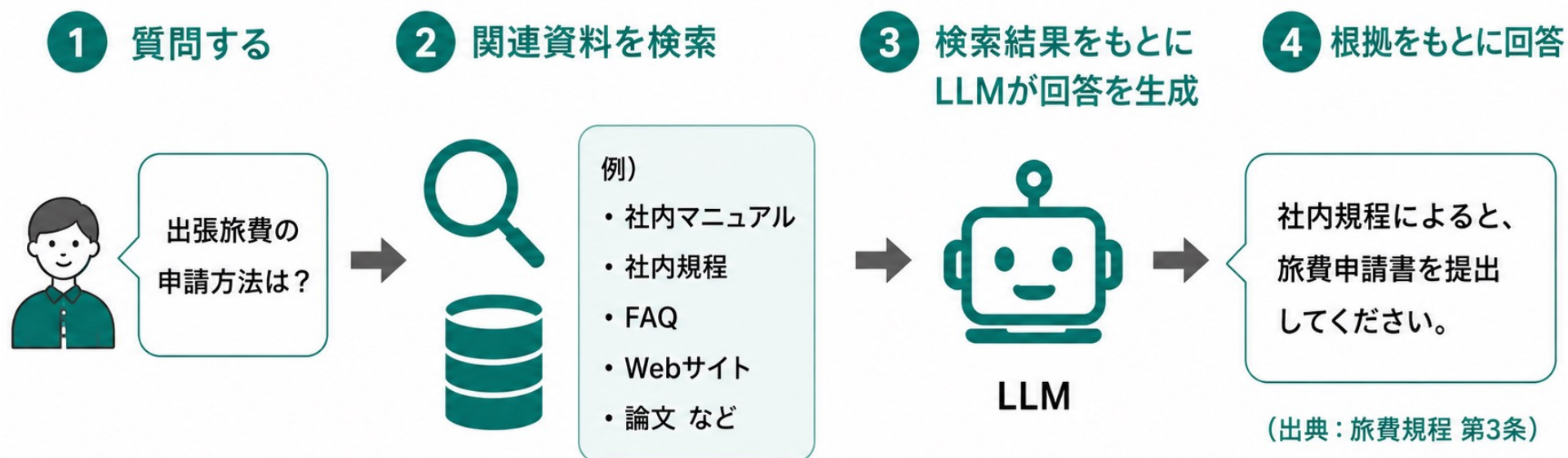
外部知識を活用し、
より正確な回答を生成する

RAGとは？

- ◆ Retrieval Augmented Generation の略
- ◆ 外部の知識ベースから関連情報を検索し、その情報をプロンプトに与えて生成
- ◆ LLMの「知らないこと」や「最新情報」の問題を解決



RAGの流れ



RAGのメリット



最新情報を
使える



社内情報を
活用できる



根拠を
示せる



ハルシネーションを
減らせる

ベクトル検索の仕組み

- ◆ ドキュメントやクエリをベクトルに変換
- ◆ ベクトルの類似度で関連ドキュメントを検索
- ◆ 一般的な手法：コサイン類似度

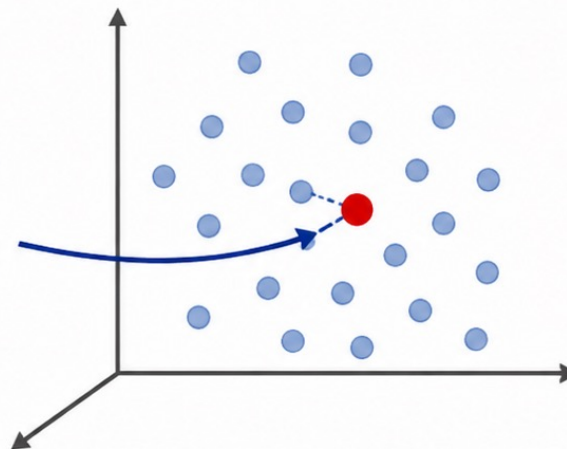
① クエリをベクトル化

クエリ（質問）を
ベクトルに変換

クエリベクトルの例
[0.12, -0.85, ...]

② ベクトル空間で検索

ベクトルの類似度（距離）で
関連するドキュメントを検索



③ 類似度の高い ドキュメントを取得

類似度が高い（スコアが高い）
ドキュメントを上位から取得

検索結果（類似度スコアの例）

📄	Doc A	0.92
📄	Doc B	0.85
📄	Doc C	0.78
	⋮	

24

RAGの効果（例）

✕ RAGなし（LLMのみ）

Q：2024年のノーベル物理学賞の受賞者は？

A：私は2024年の情報を持っていません。

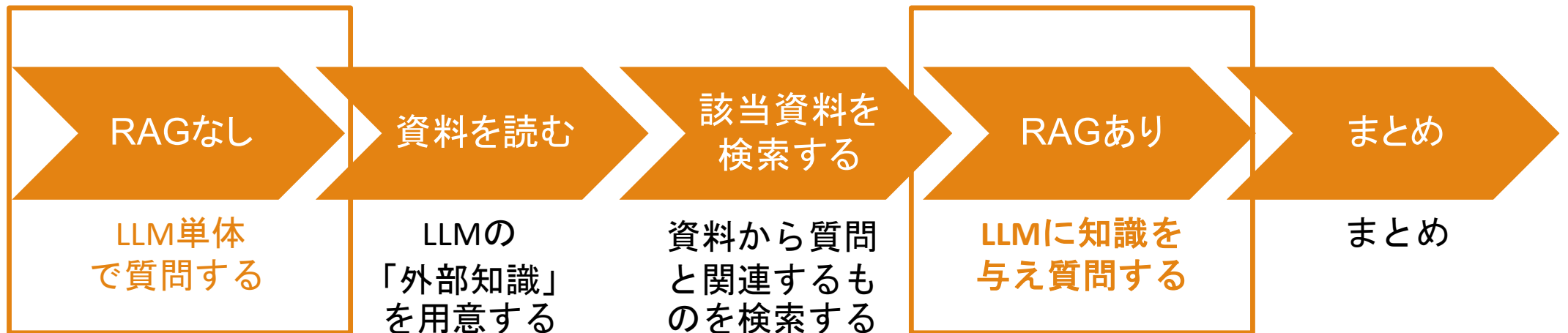


✓ RAGあり（外部情報を参照）

Q：2024年のノーベル物理学賞の受賞者は？

A：2024年のノーベル物理学賞は、ジョン・ホップフィールド氏とジェフリー・ヒントン氏が受賞しました。

RAGハンズオン



3-1

RAG-環境準備

ハンズオンのための環境を整備

Google Colabの準備



1. Google Colabへのアクセス

<https://colab.research.google.com/drive/1XdL-mBGYtATHcvzvMvYze8zDf2mhveXD?usp=sharing>

または、講習会HPから取得

03 / RESOURCES

演習・資料

パスワード入力後に、RAGとLoRAの演習リンクを表示できます。

PRIVATE MATERIALS

ハンズオン資料を表示

パスワードを入力すると、RAGとLoRAのColabリンクが表示されます。
このパスワードは画面上の表示を切り替えるためのものです。

パスワード

パスワードを入力

リンクを表示

HANDS-ON

RAGハンズオンColab

検索拡張生成の仕組みを、ノートブックで手を動かしながら確認します。
パスワードで表示

HANDS-ON

LoRAハンズオンColab

LoRAによるモデル適応を、ノートブックで実際に試しながら学びます。
パスワードで表示

PW: hi2026

Google Colabの準備



2. Googleアカウントのログイン



Googleアカウントが必要です。
未ログインの場合はログインしてください。

3. Notebookのコピー



変更を保存できませんでした

このノートブックを保存する権限がありません。変更を保存するには、ノートブックのコピーを作成してください。

キャンセル

ドライブにコピーを保存

Google Colabの準備



4. ランタイムの変更

ランタイム ツール ヘルプ

- すべてのセルを実行 ⌘/Ctrl+F9
- より前のセルを実行 ⌘/Ctrl+F8
- 現在のセルを実行 ⌘/Ctrl+Enter
- 選択範囲を実行 ⌘/Ctrl+Shift+Enter
- 現在のセルとその下のセルを実行 ⌘/Ctrl+F10

ランタイムのタイプを変更

- セッションの管理
- リソースを表示
- ランタイムログの表示

Google Cloud Run にデプロイ



ランタイムのタイプを変更

ランタイムのタイプ

Python 3

ハードウェア アクセラレータ ?

☐ CPU ☒ T4 GPU ☐ H100 GPU ☐ G4 GPU

☐ A100 GPU ☐ L4 GPU ☐ v5e-1 TPU

☐ v6e-1 TPU

i プレミアム GPU を利用するには [Colab Pro+ にアップグレードする](#)

ランタイム バージョン ?

最新 (推奨)

キャンセル 保存

Google Colabの準備



5. コードの実行

0. 準備（ここは実行するだけ）

```
#!/usr/bin/env python
#@title 0. 準備(ここは実行するだけ)
!pip -q install transformers accelerate sentence-transformers beautifulsoup4

from pathlib import Path
from urllib.request import urlopen
import numpy as np
from bs4 import BeautifulSoup
from sentence_transformers import SentenceTransformer
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
```

1回目は**時間がかかる**場合があります

トラブルシューティング



インストールが長い

- 最初の1回だけ時間がかかる

モデルの読み込みが遅い

- 初期ロードは時間がかかる
- GPU設定を確認

Web取得ができない

- ページをリロードする



③ – ②

RAG-ハンズオンの内容

実際の操作を通してRAGの実装方法を体験

Step 1: 質問を決める



question = "ヒューマンインタフェースシンポジウム2020はどこで開催されましたか？"

ポイント: 単体LLMで答えるのが難しい

Step 2: LLMの読み込み



```
llm = load_llm("sbintuitions/sarashina2.2-0.5b-instruct-v0.1")
```

SB Intuitionsが公開している**日本語特化LLM**を読み込む

※モデルをダウンロードするため、時間がかかる場合がある

<https://huggingface.co/sbintuitions/sarashina2.2-0.5b-instruct-v0.1>

Step 3: RAGなしで質問する



```
answer_without_rag = ask_without_rag(generator, question)
print("==== RAGなし ====")
print(answer_without_rag)
```

LLMの知識だけでは、回答できない



Step 4: 外部知識を用意する

```
# HI2020のウェブサイトダウンロードする
url = "http://www.his.gr.jp/sympo/his2020.html"
html_file = Path("symposium-2020.html")
urlretrieve(url, html_file)

# ダウンロードの内容を読み込む
document_text = load_text_from_html(html_file)

# 先頭500文字を確認する
print("資料の先頭:")
print(document_text[:500])
```

回答が含まれているHI2020のウェブページをダウンロードする

Step 5: 資料を小さなかたまりに分ける



```
# 資料を小さなかたまり（チャンク）に分けます。  
chunks = split_text(document_text)
```

Step 5: 資料のかたまりをベクトルに変換



```
# ベクトル変換モデルを用意する
embedding_model = load_embedding_model("sbintuitions/sarashina-embedding-v2-1b")
# 資料のかたまりをベクトルに変換する
chunk_embeddings = create_embeddings(embedding_model, chunks)
```

文書の**特徴を表すベクトル**に変換する

Step 6: 質問に関連する情報を検索する



```
# ベクトル変換モデルを用意する
embedding_model = load_emb
search_results = search_similar_chunks(
    embedding_model,
    question,
    chunks,
    chunk_embeddings,
    top_k=3, #上位3つを検索する
)
show_search_results(search_results)
```

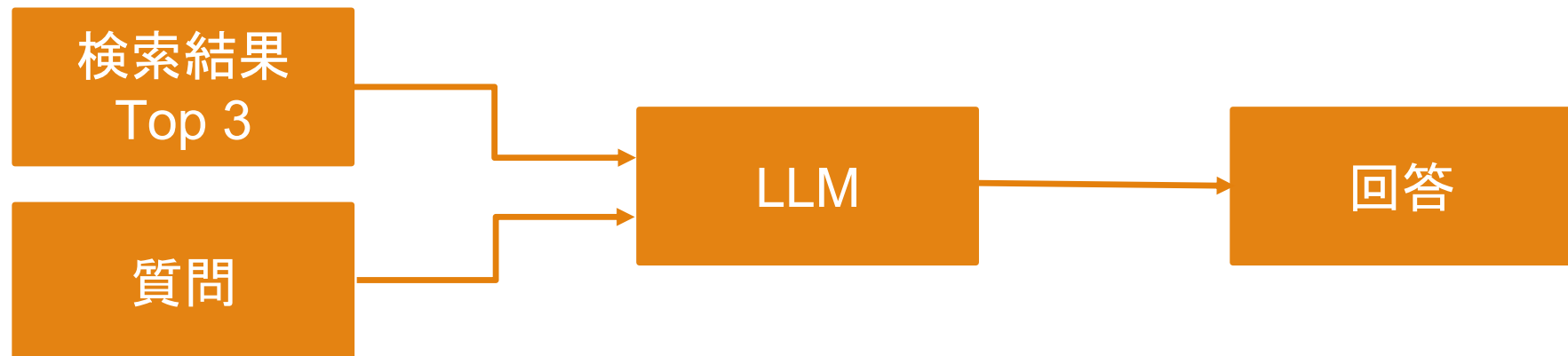
質問に**関係しそうな**（**回答できそうな**）情報を検索する

Step 6: RAGありでもう一度質問する



```
answer_with_rag = ask_with_rag(generator, question, search_results)
print("==== RAGあり ====")
print(answer_with_rag)
```

LLMに**関連情報を与え**質問する



RAGの正体



```
def ask_with_rag(generator, question, search_results):  
    context = "¥n¥n".join(item["text"] for item in search_results)
```

```
    prompt = f"""
```

あなたは、下の「資料」だけを根拠に質問へ答えるアシスタントです。
資料に書かれていないことは推測しないでください。
簡潔に答えてください。

資料:

{context}

質問:

{question}

回答:

"""

```
    return generate_answer(generator, prompt)
```

RAGの正体



元の質問文

ヒューマンインタフェースシンポジウム2020はどこで開催されましたか？

拡張された質問=RAG

あなたは、下の「資料」だけを根拠に質問へ答えるアシスタントです。
資料に書かれていないことは推測しないでください。
簡潔に教えてください。

資料:

{context} ← 関連情報

質問:

{question} ← 質問

RAGなし・ありの比較



RAGなし

1. 入力：質問のみ
2. 出力：LLMの持っている知識のみ

VS

RAGあり

1. 入力：質問 + 関連情報
2. 出力：LLMの持っている知識 + 外部知識

ポイント：質問に関連する知識の検索

まとめ



用語

- ・ チャンク/chunk: 長い資料を分けた小さな塊
- ・ Embedding : 文書の意味・特徴を表すベクトル
- ・ RAG : 外部知識をLLMに渡してから質問する

拡張

- ・ notebookの変数を変更しいろんなパターンを試す
例 : question、url、etc

4

ReACT

思考と行動を合わせて、
より高度なタスクを実行できるAIの仕組み

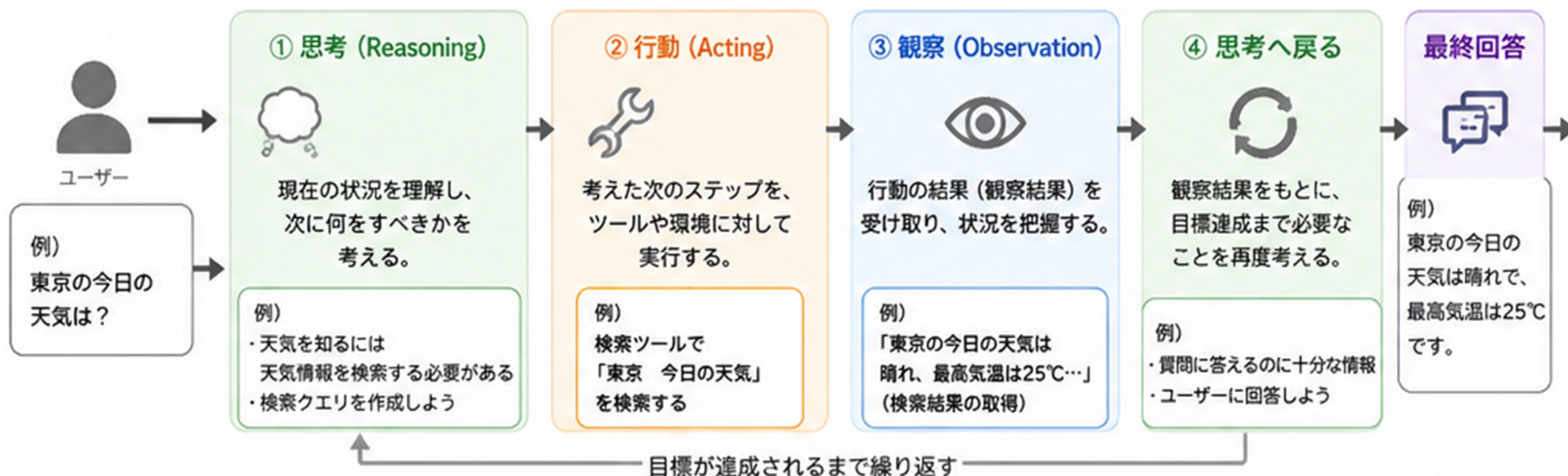
ReAct (Reasoning and Acting) とは？

LLMが「考える(Reasoning)」と「行動する(Acting)」を交互に行うことでより賢く、正確にタスクを解決するためのフレームワーク

例：天気を調べる

ReActの基本サイクル

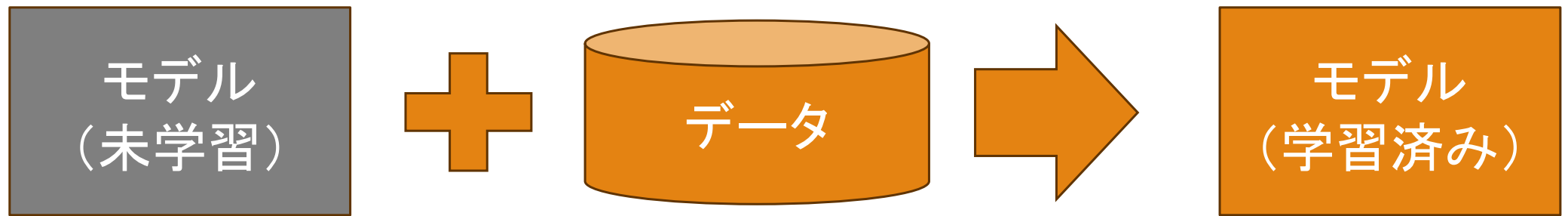
思考 (Reasoning) → 行動 (Acting) → 観察 (Observation) を繰り返す



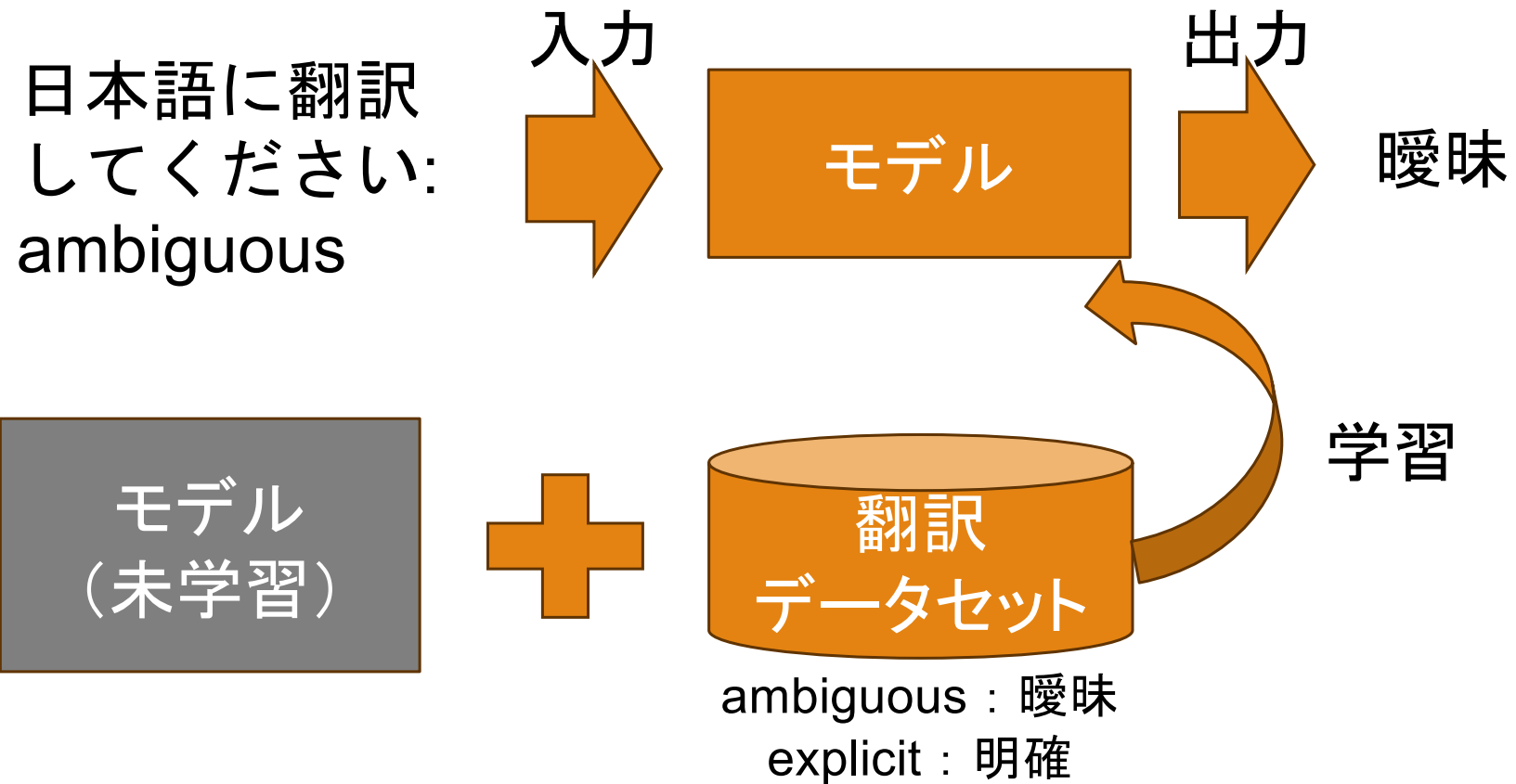
5

機械学習の基礎

モデルの学習

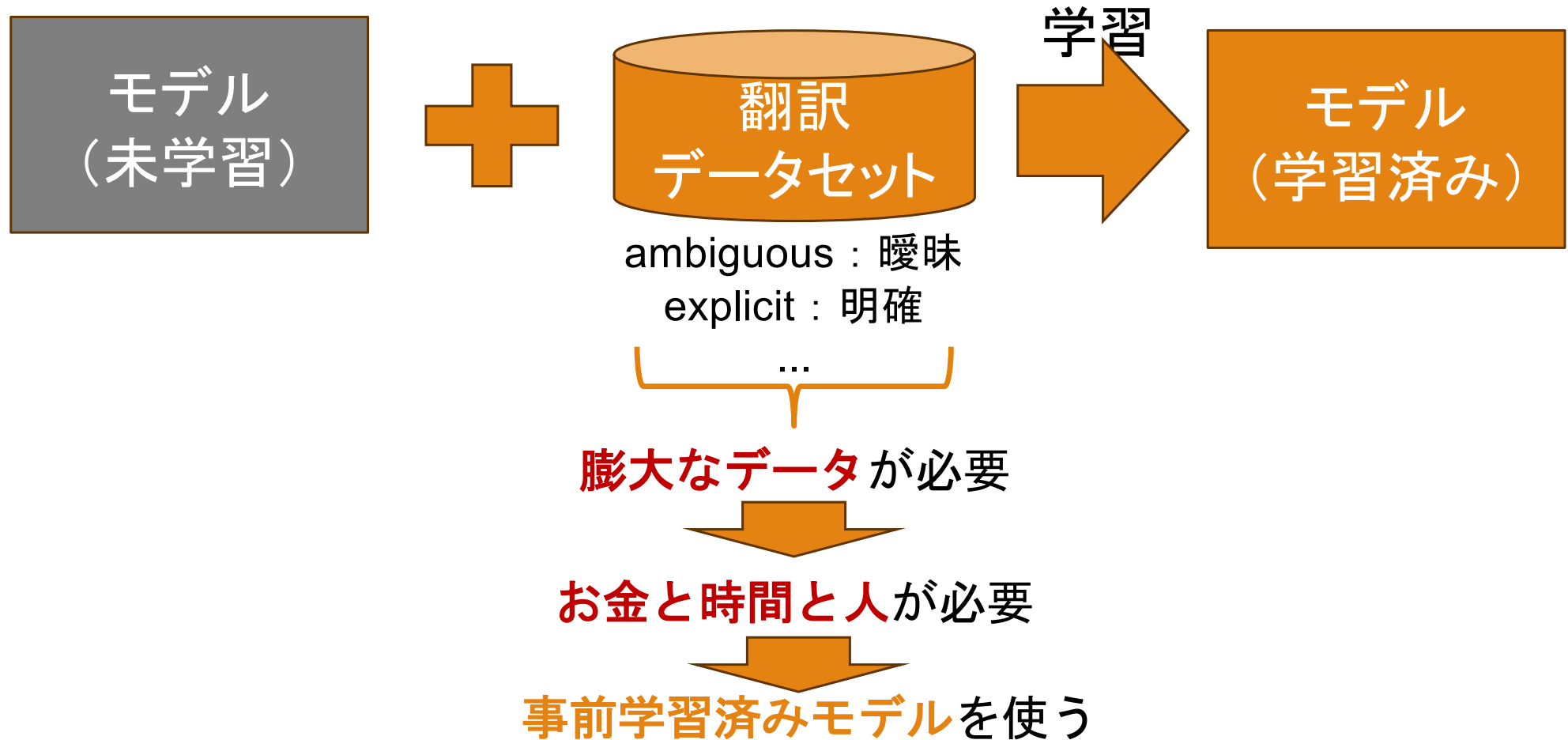


モデルの学習の例

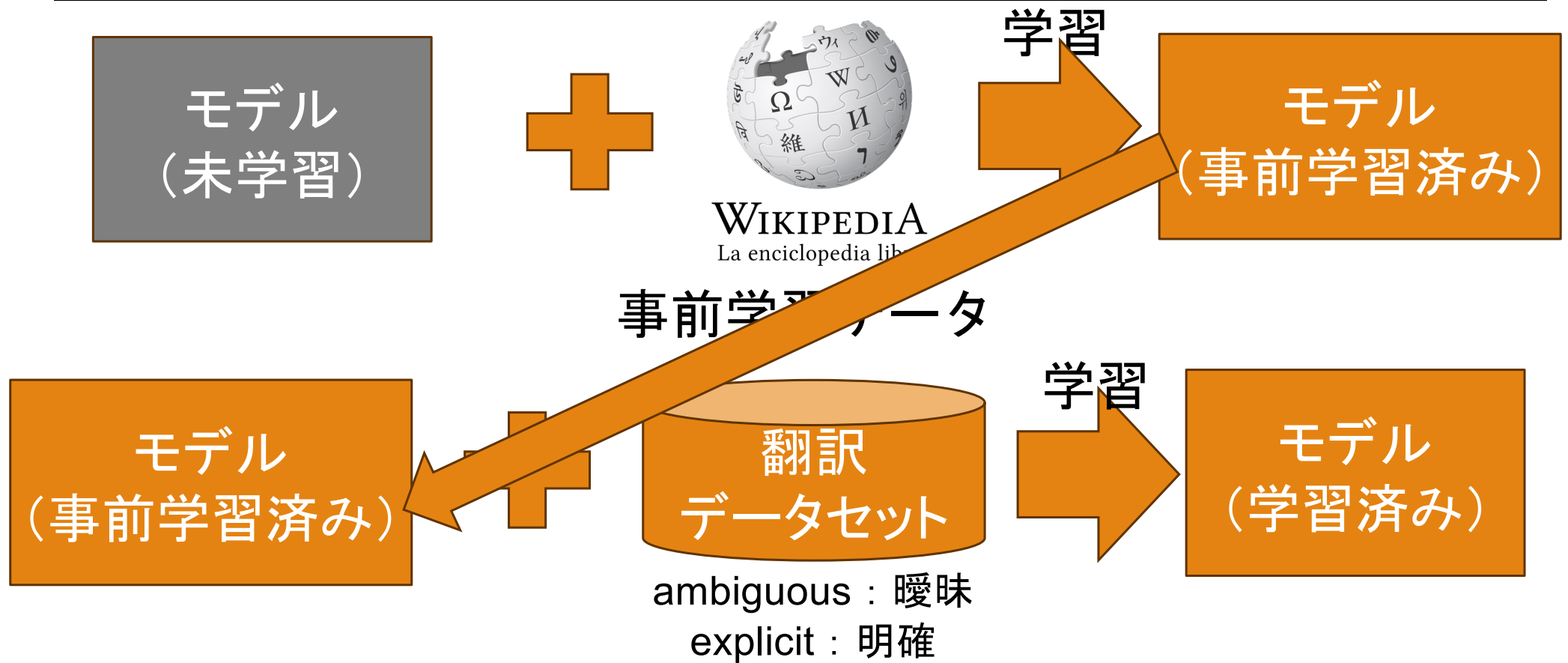


...

モデルの学習の難しさ



事前学習の仕組み



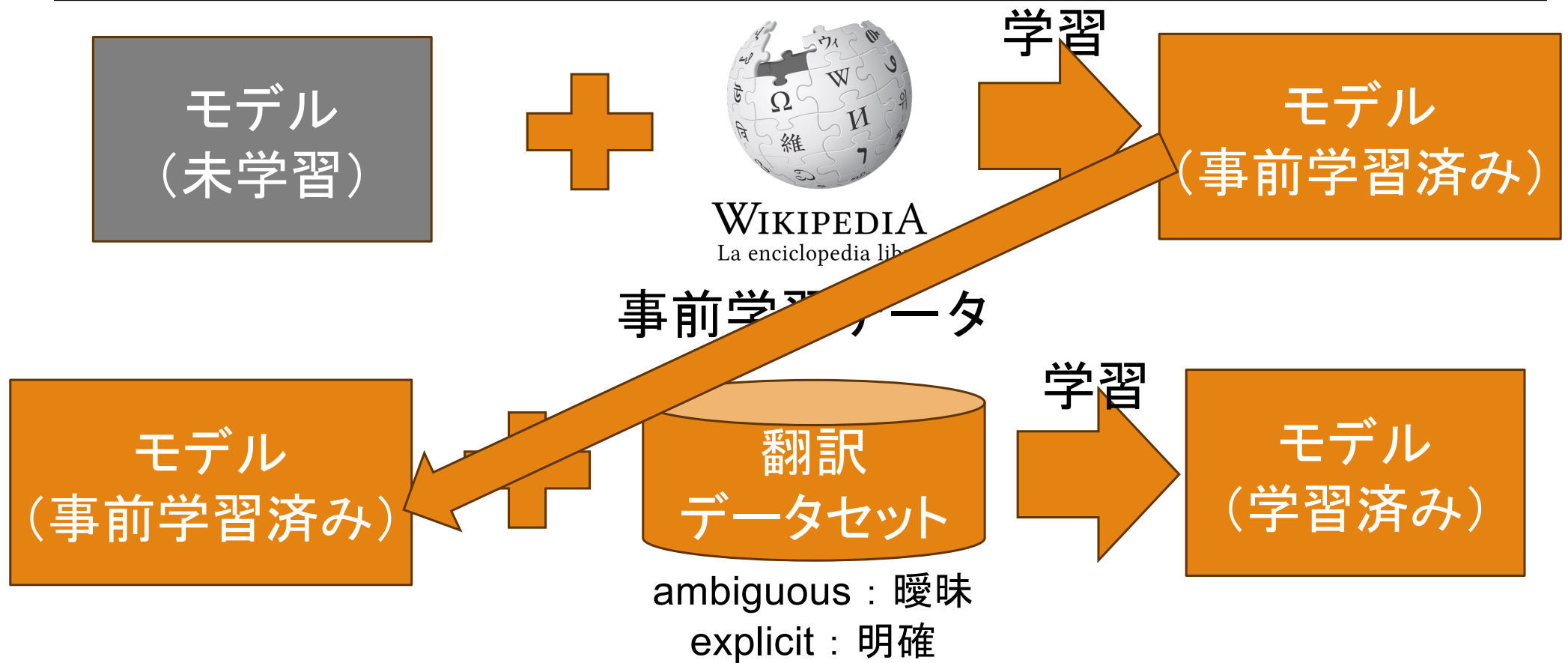
様々な事前学習済みモデル

Models 410,227 ☒ Base only ☒ Inference Available

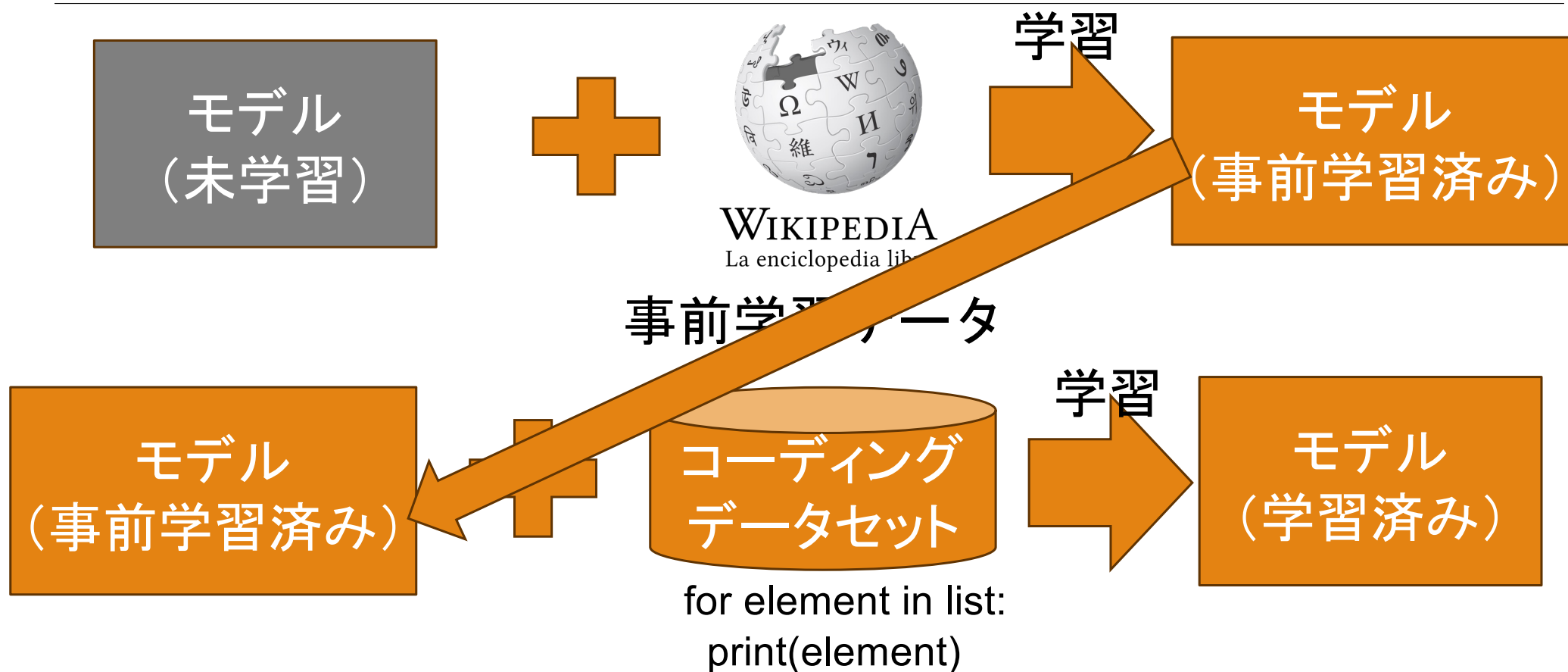
 Qwen/Qwen3-0.6B Text Generation • 0.8B • Updated Jul 26, 2025 • 20.2M • ⚡ • ❤️ 1.63k	 trl-internal-testing/tiny-Qwen2ForCausalLM-2.5 Text Generation • 2.44M • Updated 7 days ago • 16.1M • ❤️ 42
 openai-community/gpt2 Text Generation • 0.1B • Updated Feb 19, 2024 • 15.2M • ❤️ 4.08k	 unsloth/Qwen3-Coder-30B-A3B-Instruct-GGUF Text Generation • 31B • Updated Jan 30 • 12.8M • ❤️ 1.02k
 Qwen/Qwen3-8B Text Generation • 8B • Updated Jul 26, 2025 • 12.7M • ⚡ • ❤️ 2k	 Qwen/Qwen2.5-7B-Instruct Text Generation • 8B • Updated Jan 12, 2025 • 9.95M • ⚡ • ❤️ 2.21k
 nvidia/Qwen3.6-35B-A3B-NVFP4 Text Generation • 19B • Updated 17 days ago • 9.03M • ❤️ 615	 facebook/opt-125m Text Generation • Updated Sep 15, 2023 • 7.97M • ❤️ 322
 Qwen/Qwen2.5-0.5B-Instruct Text Generation • 0.5B • Updated Sep 25, 2024 • 7.91M • ⚡ • ❤️ 624	 farbodtavakkoli/OTel-2.0-LLM-31B-IT Text Generation • 31B • Updated 7 days ago • 7.43M • ⚡ • ❤️ 17

https://huggingface.co/models?pipeline_tag=text-generation&sort=downloads

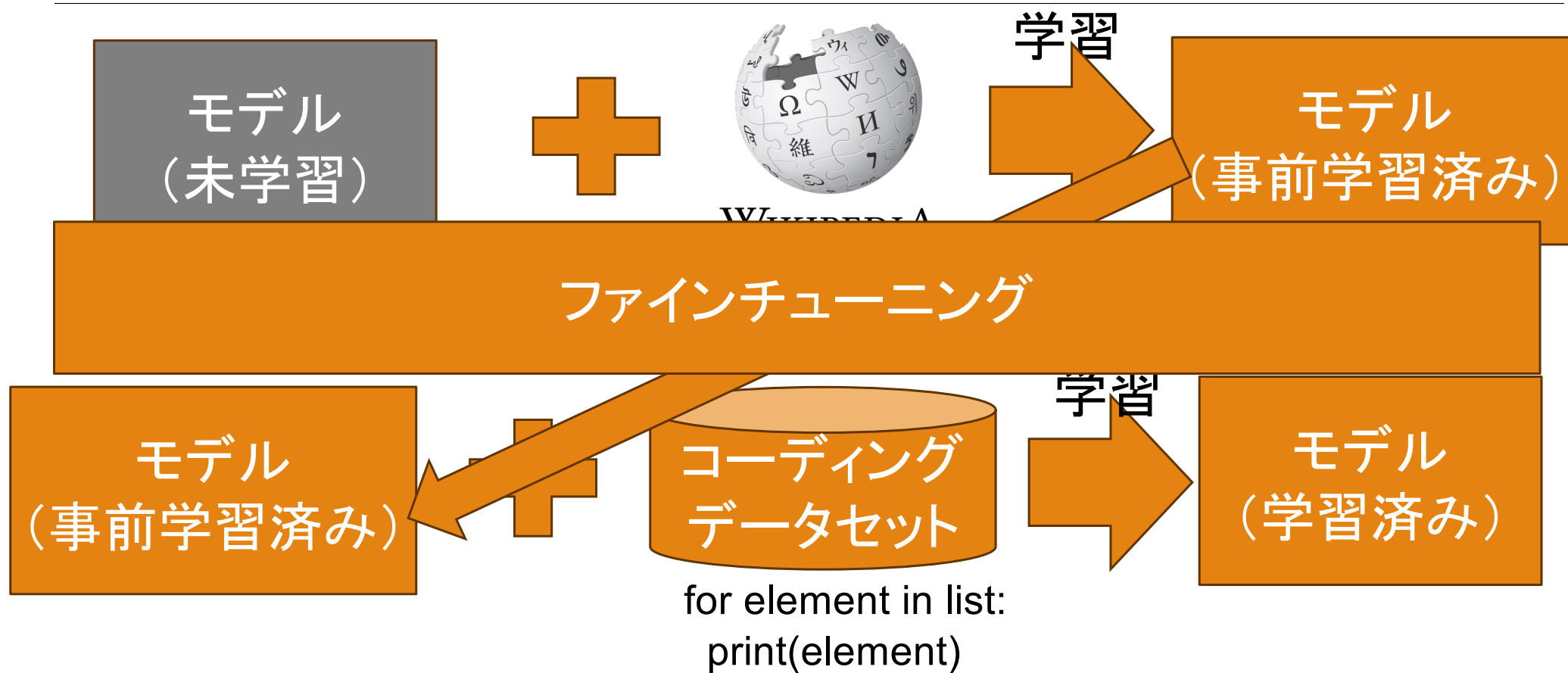
事前学習の応用：翻訳



事前学習の応用：コーディング翻訳

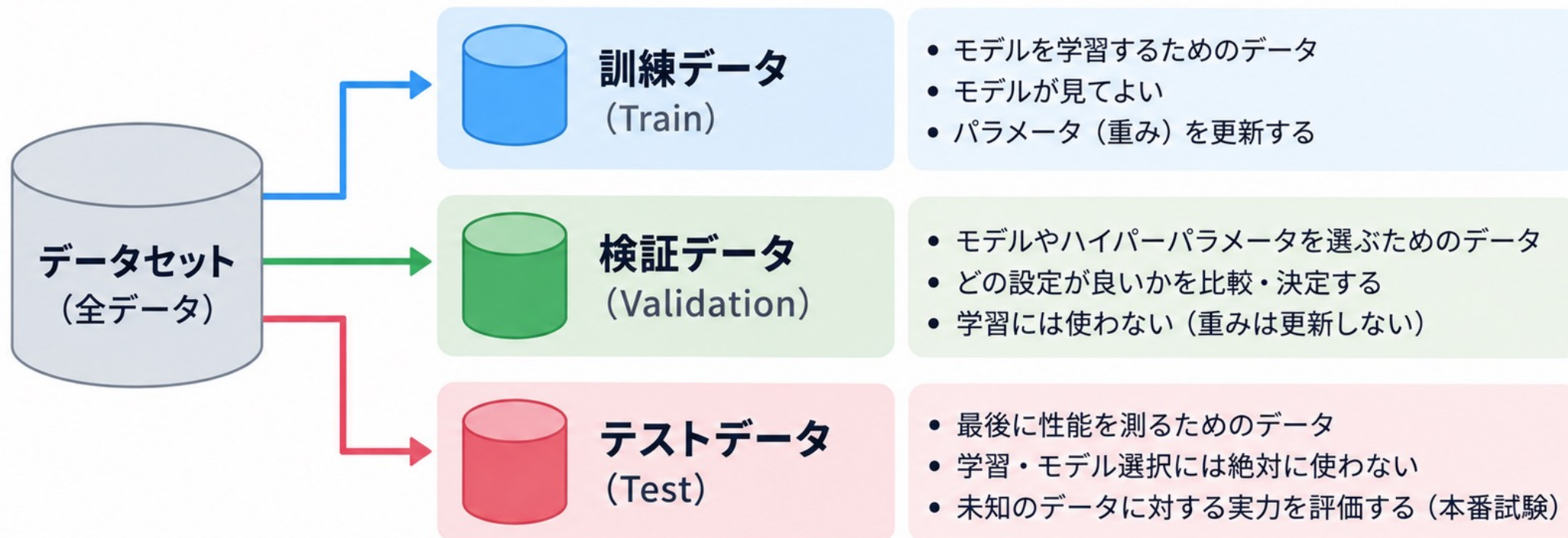


ファインチューニング



① 機械学習のお作法：データは3つに分ける

正しく評価するための基本ルール



テストデータは「本番試験」

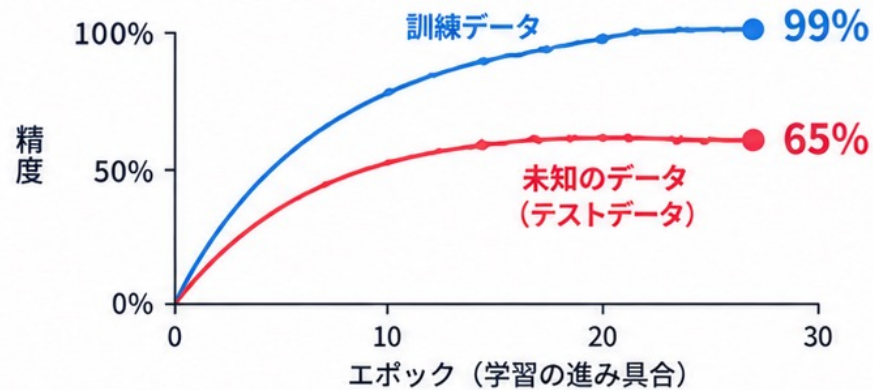
答えを見ながら勉強してはいけない！

2 学習できた ≠ 未知のデータでも解ける

検証データで適切なモデルを選ぶ

よくある失敗 訓練データだけで学習

訓練データでは高い精度が出るが、未知のデータでは性能が大きく落ちてしまう。

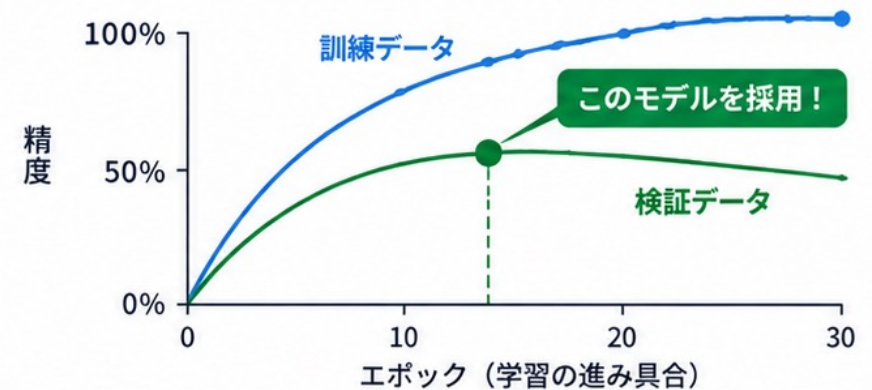


過学習 (Overfitting)

訓練データを覚えすぎて、未知のデータに対応できない。

正しい手順 検証データで確認しながら学習

検証データの性能を見ながら、最も良いモデルやパラメータを選ぶ。



検証データで性能を評価

汎化性能の高いモデルを選ぶことができる。



機械学習の目的は、訓練データを覚えることではなく、未知のデータにも対応できること。

③ データが少ないときは？ー クロスバリデーション

データを分割して繰り返し検証することで、より信頼できる評価を行う

k分割クロスバリデーションの例 (k = 5)

Fold 1	検証	訓練	訓練	訓練	訓練
Fold 2	訓練	検証	訓練	訓練	訓練
Fold 3	訓練	訓練	検証	訓練	訓練
Fold 4	訓練	訓練	訓練	検証	訓練
Fold 5	訓練	訓練	訓練	訓練	検証

- 1 データを k 個に分割する
- 2 1つを検証データ、残りを訓練データとして学習・評価を行う
- 3 これを k 回繰り返し、評価結果を平均する

すべてのデータを検証に使えるので、
1回の分割よりも安定した評価ができる



テストデータはクロスバリデーションの外側に残しておく

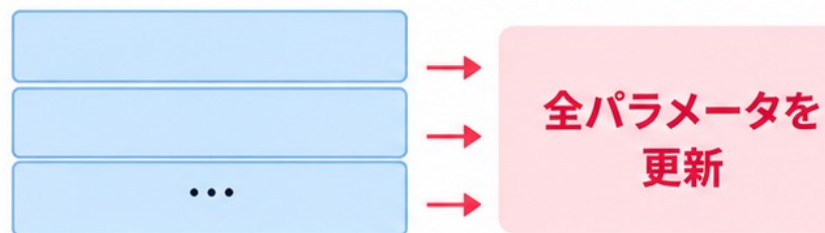
クロスバリデーションはモデル選択のために行い、最後にテストデータで一度だけ評価する。

4 モデルが大きくなった今、なぜLoRAが主流なのか？

大規模モデルの効率的な適応手法

フルファインチューニング（従来の方法）

モデルのすべてのパラメータを更新する

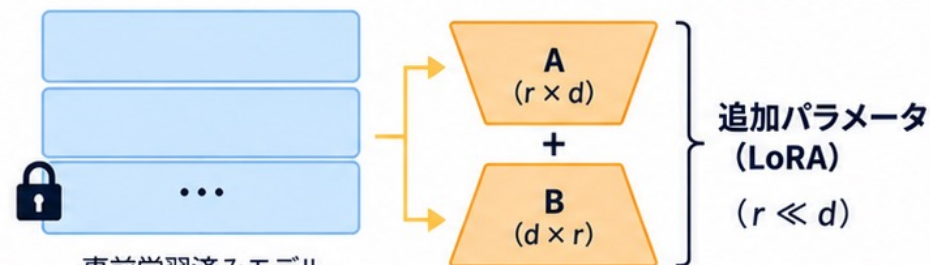


大規模言語モデル
(例: 数十億 ~ 数千億パラメータ)

- ✗ 計算コスト・メモリが非常に大きい
- ✗ 学習に時間がかかる
- ✗ 各タスクごとにモデル全体を保存する必要がある
- ✗ 大規模モデルでは現実的ではない

LoRA (Low-Rank Adaptation)

モデルの一部に小さな追加パラメータ（低ランク行列）を学習する



事前学習済みモデル
(パラメータは固定)

- ✓ 学習するパラメータが少なく、計算コスト・メモリを大幅に削減
- ✓ 短時間で学習できる
- ✓ タスクごとに小さなアダプタのみを保存すればよい
- ✓ 大規模モデルの適応手法として現在の主流



モデルのパラメータ数が急増した現在、
効率的に適応できる **LoRA** が**主流**となっている。

同じ大規模モデルを
さまざまなタスクに効率よく適応できる！

6

LoRA

効果的にLLMをチューニングし、
自分専用のLLMを構築

LoRAとは？

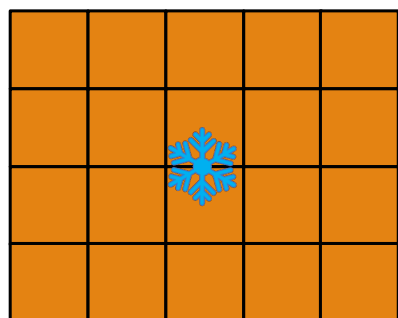
- ◆ Low-Rank Adaptationの略
- ◆ 大規模モデルのパラメータを固定し、
軽量アダプターで挙動を変える
- ◆ 少ない計算資源で効率的にファインチューニング可能

LoRA 

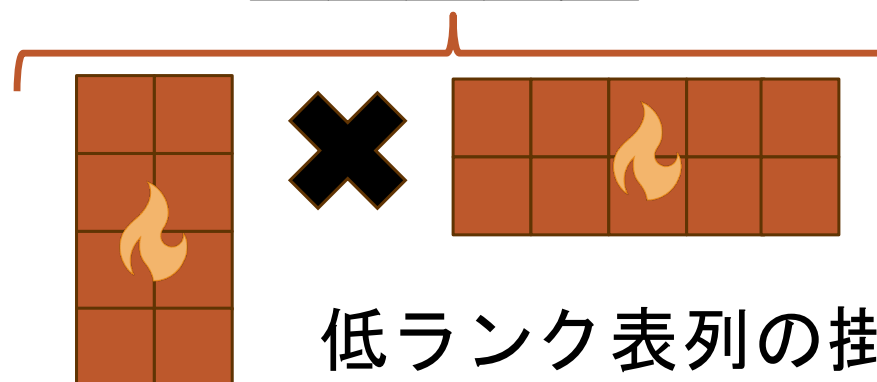
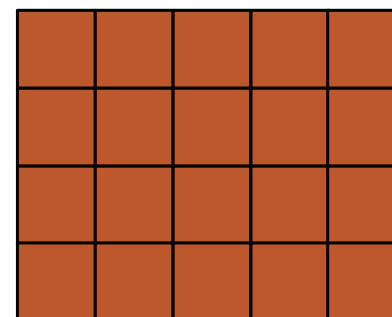
ベースの大規模モデル 

LoRAの仕組み

- ◆元の重み行列Wに、低ランク分解した更新量を加える
ベースのモデル LoRAアダプター



学習しない



低ランク表列の掛け算

LoRAのメリット

ファインチューニングの**数%のコスト**で実現！

学習パラメータが大幅に削減される

GPUメモリ使用量を抑えられる

学習が高速でコストが低い

失敗のタスクに対してアダプタを使い分け可能

学習の流れ

1. データセットの準備・トークン化

2. ベースモデルの読み込み

3. LoRA設定の作成

4. LoRAアダプタを付与

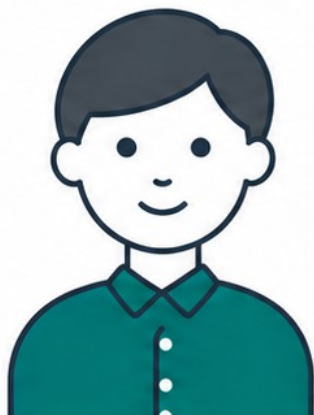
5. 学習(アダプタのみ更新)

6. アダプタの保存・推論

1

Reviewer #2 LoRAチュートリアル

～ 30分で「辛口な査読者」をつくってみよう～



私の研究、
Reviewer #2 だったら
なんて言うだろう…？



このチュートリアルでは、
日本語で辛口なレビューを
行うLLM（Reviewer #2
LoRA）を、LoRAを使って
作成します。

LoRAを使うと、既存のLLMに小さな追加パラメータだけを学習させて、
特定の役割やスタイルを持つモデルを作ることができます。

今回は、学术论文に対して「**厳しめの査読コメントを返すReviewer #2**」を
作ってみましょう！

2

まずは普通のLLMに査読させてみよう

～ そのままでは優しすぎる？～



入力（日本語のAbstract）

本研究では、オンライン会議における参加者の非言語的な反応を推定する手法を提案する。発話・視線・表情などのマルチモーダル情報を用いて、会議の参加度を推定し、実験により提案手法の有効性を示した。



Base LLM の出力



一般的なLLM

本研究は、オンライン会議における参加者の行動理解という重要な課題に取り組んでおり、興味深い研究です。提案手法も妥当であり、今後の発展が期待されます。



たしかに良いコメントだけど、
これってReviewer #2っぽくないかも…？



ここがポイント！

既存のLLMは、丁寧でバランスの取れたレビューを生成しますが、批判的な指摘や厳しい評価はあまり出力しません。

そこで、LoRAを使って「Reviewer #2」のスタイルを学習させます。

3

今回作るのはこんな Reviewer #2

～ 厳しめ・具体的・建設的 (?) な査読コメント ～



この論文、
いくつか重大な問題が
あります…



Reviewer #2



Reviewer #2 の特徴

- ✓ 論文の問題点を率直かつ具体的に指摘する
(良い点だけでなく、欠点も明確に述べる)
- ✓ 評価観点に基づいた構成でコメントする
(動機・新規性・比較・実験・主張と根拠 など)
- ✓ 最終的に明確な判定を行う
(Accept / Weak Accept / Weak Reject / Reject)



出力の例

【総評】

問題設定は興味深いものの、現状では主張を十分に支持するだけの評価が不足しています。

【主要な懸念】

1. 既存研究との惹分が不明確です。
2. 実験設定・評価指標が不十分です。
3. 主張に対する根拠が弱いです。

【判定】

Weak Reject

このような「**厳しめの査読コメント**」を返す **Reviewer #2** を
LoRA を使って作成します。

4

全体の流れ

～ 4つのステップで Reviewer #2 をつくります ～



1 データ準備



日本語のAbstractと
査読コメントの
ペアデータを用意する

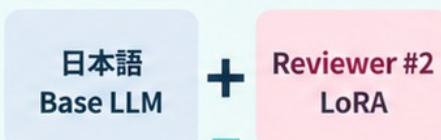
2 LoRA学習



QLoRAで
Reviewer #2 の
スタイルを学習する

- ✓ 4-bit 量子化で効率的に学習
- ✓ LoRA のみを学習 (PEFT)
- ✓ Abstract部分は損失に含めない

3 モデルに適用



Reviewer #2
(査読者LLM)

学習したLoRAを
既存のLLMに適用する

4 評価・実験



- ✓ 未知のAbstractで動作確認
- ✓ Base LLMとの出力を比較
- ✓ さまざまなAbstractで評価

厳しめの査読コメントを返す
Reviewer #2 の完成！

Google Colabで、最初から最後まで実際に手を動かして体験します！

5 データセットの準備

～ 日本語のAbstractと査読コメントのペアを用意しよう ～



まずは学習に使う
データを準備しましょう！
日本語の論文を集めて、
Abstractと査読コメントのペアを作ります。



HI2026 LoRA



学習データの例 (1件)

入力：Abstract

本研究では、VR空間における
共同作業を支援する新しい
ポインティング手法を提案する。…

出力：Reviewer #2 の査読コメント

【総評】

着想は理解できるが、現状では…

【主要な懸念】

1. 既存研究との益分が曖昧
2. …

【判定】 Weak Reject



データの集め方



1. 論文を収集

日本語の学術論文
(例：J-STAGE など)
から収集



2. ペアを作成

Abstract と
査読コメントを
対応付ける



3. 前処理

- ・不要な情報を削除
- ・フォーマットを統一
- ・テキストを整形



データのポイント



日本語の論文・査読コメントを使用



複数の分野・トピックを含める
(汎化性能の向上)



役割・評価語・指摘のバランスを考慮



今回はチュートリアル用に少数
(例：10～20件) でも十分



良いデータセットが、良いLoRAの第一歩です！

今回は少量のデータでも学習可能なので、まずは気軽に作ってみましょう。

6

QLoRAで学習する

～ 少ない計算資源でも学習できる設定 ～



ここでは、
日本語LLMにLoRAを追加して
Reviewer #2 のスタイルを
学習してみましょう！



学習の構成



日本語LLM
(例：1.8B)



LoRA
(小さな追加パラメータ)
のみ学習

4-bitで読み込み
(重みは固定)

QLoRAとは？

4-bit量子化で基盤モデルを読み込み、
LoRA のみを学習する効率的な手法です。
少ないGPUメモリでもSFT (教師あり学習) が可能です。



学習の設定例 (Google Colab)



ベースモデル

llm-jp/llm-jp-3-1.8b-instruct



量子化

4-bit (bitsandbytes)



学習方法

QLoRA (PEFT)



学習データ

日本語のAbstract + 査読コメント



LoRAの設定

rank: 16, α : 32, dropout: 0.05



学習時間の目安

数分～十数分 (ColabのGPUでOK)



Google Colab で実際に学習してみましょう！

すぐに実行できるノートブックを用意しています。

7

学習したLoRAをモデルに適用

～ 同じLLMにLoRAを追加するだけで、別の人格に～



学習が完了したら、
ベースのLLMにLoRAを適用して
Reviewer #2 のスタイルで
生成してみましょう！



LoRAの適用方法 (Google Colab)

1

ベースモデルを読み込む

学習時と同じ日本語LLMを使用

```
model = AutoModelForCausalLM.from_pretrained(...)
```

2

学習したLoRAを読み込む

PEFTのfrom_pretrained でLoRAを適用

```
model = PeftModel.from_pretrained(model, ...)
```

3

推論設定を行う

生成パラメータ (温度、最大トークン数など) を設定

4

生成してみる

同じプロンプトでも、LoRAを適用するだけで出力が変化！

```
output = model.generate(...)
```



生成結果の比較

Base LLM の出力 (Before)

本研究は興味深いテーマを扱っており、
オンライン会議における非言語情報の
活用について検討しています。
提案手法は一定の有効性が期待でき、
今後の発展が望まれます。



やや甘めのコメント

Reviewer #2 LoRA の出力 (After)

本研究は興味深い試みですが、
いくつかの重要な課題が残っています。
実験設定の妥当性や評価指標の選択に
ついて、より詳細な検討が必要です。
また、関連研究との比較が不十分であり、
主張の新規性が明確ではありません。



厳しめのコメント



適用のポイント

- ✓ 学習時と同じベースモデルを使用
- ✓ 学習したLoRAのパスを正しく指定
- ✓ 推論時のプロンプトは同じにして比較
- ✓ LoRAを適用するだけで出力スタイルが変化
- ✓ 複数のプロンプトで動作を確認
- ✓ 必要に応じて生成パラメータを調整

8

実際に査読させてみよう

～ 同じ Abstract で、Before / After を比較 ～



同じAbstractを
Base LLM と Reviewer #2 LoRAで
査読させてみましょう！
どんな違いがあるのでしょうか？



Base LLM の出力 (Before)

入力：Abstract

本研究では、オンライン会議における参加者の非言語的な反応を推定する手法を提案する。…



出力：査読コメント (例)

【総評】

興味深い取り組みです。
提案手法は妥当であり、今後の
応用が期待されます。…



Reviewer #2 LoRA の出力 (After)

同じ入力：Abstract

本研究では、オンライン会議における参加者の非言語的な反応を推定する手法を提案する。…



出力：査読コメント (例)

【総評】

着想は理解できるが、現状では …

【主要な懸念】

1. 既存研究との差分が曖昧
2. 評価指標の妥当性について説明が不足

【判定】 Weak Reject



Reviewer #2



データのポイント



同じAbstractを使用



厳しめの査読コメントを
生成（より詳細な指摘）



役割・評価基準・出力形式を
そろえて比較



今回はチュートリアル用に
少数（例：10～20件）でも十分

同じモデル・同じプロンプトでも、LoRAを付けるだけで出力のスタイルが大きく変わります！

まとめ

プロンプト設計

RAG(ハンズオンあり)

ReAct

機械学習の基礎知識

LoRA(ハンズオンあり)